Heap Data Structures: Implementation, Analysis, and Applications

1. Heapsort Implementation and Analysis

The heaps are tree based data structures that maintains a hierarchical ordering property, where in a max-heap the parent nodes are all greater than or equal to children and in a min-heap the parent nodes are all less than or equal to children. Such properties render heaps useful especially in dealing with applications like priority queue, scheduling, and efficient sorting algorithms like Heapsort.

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heapsort(arr):
    n = len(arr)
    for i in range(n//2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
    return arr

# Example usage
arr = [4, 10, 3, 5, 1]
sorted_arr = heapsort(arr)
print(sorted_arr)
```

It starts by processing the input array into a max-heap, which makes sure that the biggest element is at the root. After building the heap, the algorithm switches the root and the final element of the heap repeatedly and decreases the heap size by one repeatedly and reestablishes the max-heap property by a heapify algorithm. This will be repeated till all elements has been extracted leaving a sorted array in ascending order. The two important operations in Heapsort, the heap building and the heapifying during extraction determine the time complexity of Heapsort.

As it is shown in the theoretical analysis, Heapsort is characterized by the best, average, and worst time complexities of O(n log n). Building a max-heap out of n items requires O(n) time and pulling out each item of the heap uses O(log n) time to pull out one item, and O(n log n) time is used to pull out all items in the heap. O(1) space complexity of the in-place Heapsort with another minor consumption of memory by the recursion stack in the process of heapify with O(log n) additional space availability. This is hypothetical performance, which implies that it means that Heapsort will behave similarly no matter the input order and so is a reliable choice in applications whereby predictable behavior is essential.

Experiment Result:

Dataset	Size	Heapsort	Quicksort	Merge Sort	Python Sort
Random	1000	0.001251s	0.000744s	0.000909s	0.000068s
	5000	0.007976s	0.004379s	0.005496s	0.000404s
	10000	0.017803s	0.009429s	0.011795s	0.000853s
Sorted	1000	0.001861s	0.000443s	0.000611s	0.000002s
	5000	0.008444s	0.002724s	0.003538s	0.000010s
	10000	0.018421s	0.005733s	0.007626s	0.000020s
Reverse Sorted	1000	0.001138s	0.000444s	0.000600s	0.000003s
	5000	0.007112s	0.002536s	0.003454s	0.000010s
	10000	0.015333s	0.005468s	0.007273s	0.000019s

Those theoretical predictions are supported by empirical experiments. Heapsort took about 0.001251s, 0.007976s and 0.017803s respectively to sort random datasets of sizes of 1000, 5000 and 10000 respectively. In the case of sorted datasets, the times were 0.001861s, 0.008444s, and 0.018421s when the data is sorted and in the reverse-sorted case, the times were 0.001138s, 0.007112s, and 0.015333s. These findings demonstrate the performance of the algorithm is stable regardless of the distribution of inputs. Comparatively, Quicksort was quicker with randomized data since it was able to make good pivot selections, but slower with sorted or reverse-sorted data, which is in accordance with its worst-case theoretical complexity of O(n 2). Merge Sort was always the same and this was because it follows the complexity of O(n log n), however it needs extra memory space that is equivalent to the size of the array. The internal optimizations of Python inbuilt Timsort was overall better than the three implementations, although it is not a pure heap-based sort. In general, the empirical findings indicate that the runtime of Heapsort is consistent, suggesting the accuracy of the algorithm and predictability in terms of its speed in order to sort large data set or system on a worst-case basis.

Priority Queue Implementation and Applications

1. Introduction

Task scheduling is a very important particularly when the tasks are of varying priority, arrival time as well as deadline. I have implemented a scheduler that is a priority based queue which emulates a token based system in which tasks will be served depending on their priority.

Every task in the scheduler is the instance of Task class. The Task class contains all the specific data to be used in planning and assigning priorities: the task ID that is unique and the priority, the time of arrival, and the time of deadline.

- task_id: a unique identifier
- priority: numeric value representing urgency (lower value = higher priority)
- arrival_time: the time at which the task enters the system
- deadline: the latest acceptable time to serve the task

This design enables a realistic simulation of task management systems.

```python
class Task:
    """
    Represents a task with:
    - task_id: unique identifier
    - priority: lower value = higher priority
    - arrival_time: when task enters system
    - deadline: task completion deadline
    """

    def __init__(self, task_id, priority, arrival_time, deadline):
        self.task_id = task_id
        self.priority = priority
        self.arrival_time = arrival_time
        self.deadline = deadline

    def __lt__(self, other):
        """
        Defines comparison for heap:
        - First compare priority
        - If equal, compare arrival time (FIFO behavior)
        """
        if self.priority == other.priority:
            return self.arrival_time < other.arrival_time
        return self.priority < other.priority

    def __repr__(self):
        """
        String representation for debugging
        """
        return (f"Task({self.task_id}, priority={self.priority}, "
                f"arrival={self.arrival_time.strftime('%H:%M:%S')}, "
                f"deadline={self.deadline.strftime('%H:%M:%S')})")
```

The __lt__method can make sure that the heap will sort tasks using predominantly priority, and in case of a tie, the arrival time. This design option ensures that tasks of high urgency (lower numeric priority) are served first, and FIFO is followed among tasks of the same priority.

2. Priority Queue Implementation

Priority queue written in Python with the help of the heapq module offers a fast binary min-heap. A min-heap is appropriate since lower the number of the priority of a task the higher is the priority.

```python
class PriorityQueue:
    """
    Implements a priority queue using a binary min-heap.
    """

    def __init__(self):
        self.heap = []                  # underlying heap
        self.entry_finder = {}          # maps task_id -> task object

    def insert(self, task):
        """
        Insert a task into the priority queue.
        Time Complexity: O(log n)
        """
        heapq.heappush(self.heap, task)
        self.entry_finder[task.task_id] = task

    def extract_min(self):
        """
        Remove and return highest priority task (lowest value).
        Time Complexity: O(log n)
        """
        while self.heap:
            task = heapq.heappop(self.heap)
            if task.task_id in self.entry_finder:
                del self.entry_finder[task.task_id]
                return task
        return None

    def change_priority(self, task_id, new_priority):
        """
        Update the priority of a task.
        Time Complexity: O(n) + O(log n)
        """
        if task_id not in self.entry_finder:
            print(f"Task {task_id} not found!")
            return

        task = self.entry_finder[task_id]
        old_priority = task.priority

        # Remove and reinsert to maintain heap property
        self.heap.remove(task)      # O(n)
        heapq.heapify(self.heap)    # restore heap after removal

        task.priority = new_priority
        heapq.heappush(self.heap, task)  # O(log n)

        print(f"Priority of {task_id} changed from {old_priority} → {new_priority}")

    def is_empty(self):
        """
        Check if queue is empty.
        Time Complexity: O(1)
        """
        return len(self.entry_finder) == 0
```

Operation	Complexity	Notes
insert(task)	O(log n)	Inserts task into min-heap
extract_min()	O(log n)	Removes highest-priority task
change_priority()	O(n) + O(log n)	Remove and re-insert to maintain heap
is_empty()	O(1)	Checks if queue is empty

The combination of heap operations and a task ID mapping allows efficient dynamic scheduling.

3. Task Simulation

To demonstrate the scheduler, I have created a fixed set of 15 tasks with varying priorities, arrival times, and deadlines. Some of the task has same Arrival time and Priorities. All timestamps are in HH:MM:SS format.

Initial Task List:

```
Initial Task List:
Task ID Priority Arrival    Deadline
---------------------------------------------
T1      5        08:00:00   10:15:23
T2      9        08:00:00   11:00:00
T3      3        08:30:00   12:00:00
T4      7        08:45:00   11:45:00
T5      6        09:00:00   13:00:00
T6      8        09:15:00   12:30:00
T7      4        09:30:00   14:00:00
T8      10       09:30:00   11:15:00
T9      2        10:00:00   15:00:00
T10     1        10:15:00   16:00:00
T11     5        10:15:00   12:30:00
T12     7        10:45:00   13:15:00
T13     6        11:00:00   14:30:00
T14     9        11:30:00   13:00:00
T15     3        12:00:00   15:15:00
```

- Tasks are inserted sequentially into the queue.
- Priority alone determines extraction order, but arrival time resolves ties.

4. Scheduler Simulation Logic

The scheduler simulation demonstrates task serving behavior:

1. Extract top 3 highest-priority tasks (lowest numeric value).
    a. T10 (1), T9 (2), T15 (3) are removed first.
2. Dynamic priority changes:
    a. T8's priority is changed from 10 → 1, making it the highest priority remaining task.
    b. The system prints a message when a task's priority changes.
3. Final Priority Queue:
    a. Remaining tasks are sorted by priority to show current service order.

```
Extracting top 3 highest priority tasks (lowest value = highest priority):
T10 (Priority: 1, Deadline: 16:00:00)
T9 (Priority: 2, Deadline: 15:00:00)
T3 (Priority: 3, Deadline: 12:00:00)
Priority of T8 changed from 10 → 1
```

Final Priority Queue (sorted by priority):

```
Final Priority Queue (Sorted):
Task ID Priority Arrival    Deadline
---------------------------------------------
T8      1        09:30:00   11:15:00
T15     3        12:00:00   15:15:00
T7      4        09:30:00   14:00:00
T1      5        08:00:00   10:15:23
T11     5        10:15:00   12:30:00
T5      6        09:00:00   13:00:00
T13     6        11:00:00   14:30:00
T4      7        08:45:00   11:45:00
T12     7        10:45:00   13:15:00
T6      8        09:15:00   12:30:00
T2      9        08:00:00   11:00:00
T14     9        11:30:00   13:00:00

Is queue empty? False
```

- is_empty() returns False : tasks remain in the queue after partial extraction.

5. Observation

- Priority-Based Task Serving : The priority is done in the form of a min-heap where a task whose priority is lower in number is deemed to be of higher urgency. In a binary heap, the root node is always the minimum element. Therefore, the O(log n) time extraction

(extract_min) can be used to extract the highest priority task at any time. For example, T10 (priority 1) is attended to first, even when other tasks arrived earlier. This is a real-life scheduling situation in which there is no emphasis on arrival time as there is urgency.

- Tie-Breaking Using Arrival Time: When there are two tasks of same priority, the scheduler performs the one with the earlier arrival time. This is introduced in __lt__ method where arrival time is compared when the priorities are same. It guarantees a FIFO behaviour of equally urgent tasks. T3 and T15 are both in priority 3 however, T3 is served first since it arrived earlier. This is vital in real time systems to avoid the starvation of tasks that have the same urgency.
- Dynamic Priority Changes: Tasks can have their priorities updated after insertion. The task is removed from the heap (O(n)) and reinserted (O(log n)) making sure that the property of the heap is preserved. When the priority of T8 is adjusted to 1 from 10, it instantly becomes the highest priority task that has not been served, which proves to be flexible in handling dynamically run tasks. This is the modeling of real-world situations, like when there are emergency customer requirements or important processes within the system, where the priorities may change depending on the circumstances.
- Task Extraction and Queue Behavior: Tasks extracted are forever deleted off the queue, so that they are not served twice. Tasks which have not been served are only in the final queue. This division guarantees clarity of the tasks which are already done or those which are pending. The top 3 tasks (T10, T9, T15) are eliminated and the rest of the tasks are reconnected in terms of their priority which accounts the dynamic change of the queue.
- Deadline Awareness and Fairness: Although the current implementation is based on priority, it can be expanded to deal with deadline expirations. Tasks in real systems that fail to meet their time limits might require special handling (e.g. rescheduling or notification). This makes sure that nothing is left behind, which offers fairness in scheduling.
- Time Representation for Clarity: HH:MM:SS format allows humans to fast perceive arrival and deadline time, which is essential in reports or real-time monitoring system. Facilitates debugging, and also enables visualization of the overlapping tasks and evolution of queues during simulations.
- Dynamic Queue Evolution: The priority queue updates as tasks arrive dynamically. At each time step, new tasks are inserted, and the heap reorders automatically. In operating systems, this is a model of processes that come in at different times and are scheduled without interrupting the system. The scheduler is able to deal with overlapping arrivals in a graceful manner and has the right priorities.

6. Time Complexity Analysis

- Insertion (insert): The efficiency of the scheduler is closely tied to the underlying priority queue operations. The insertion (insert) operation of a binary heap entails the insertion of

the task at the tail of a heap and then bubbling up, to reestablish the heap property. A binary heap of n elements is approximately $\log_2(n)$, thus the maximum amount of comparisons and swaps needed in an insertion operation is O(log n). This ensures that with the increase in number of tasks, addition of new tasks can be fast thus rendering the scheduler dynamic to changes in the environment.

- Extraction (extract_min): The extraction operation (extract_min) is also efficient. By removing the root of the heap and then "bubbling down" the last element to restore heap structure, the scheduler retrieves the highest-priority task in O(log n) time. It will make sure that urgent jobs have been served in time and this is essential in real time based systems or high throughputs systems where completion of tasks in a timely fashion is paramount.
- Checking for emptiness (is_empty) and peeking (peek): Checking whether the queue is empty (is_empty) and inspecting the next task without removal (peek) are O(1) operations. These are very light and are utilized to manage the simulation loops, whether or not to proceed the simulation time in addition to tracking the task that is to be served subsequently. They are characterized by the fact that their time complexity is always small hence their overhead is minimal even when the queue has a large number of tasks.
- Adjusting Task Priority: A slightly less efficient operation in the current Python implementation is priority adjustment (change_priority). Since the standard heapq module does not support direct element updates the task must be removed from the heap which is an O(n) operation, and then reinserted O(log n) to maintain heap order. Although this is fine when the number of tasks in a simulation is moderate, where time-performance or performance is needed in large-scale or performance-critical applications, it is possible to maintain a mapping of task index in the heap, which would enable priority updates in the system in O(log n) time, thus making the system more scalable.

7. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Python Official Documentation: https://docs.python.org/3/
- Github: https://github.com/sranabhat51358/MSCS532_Assignment4.git